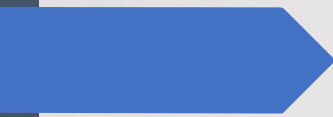


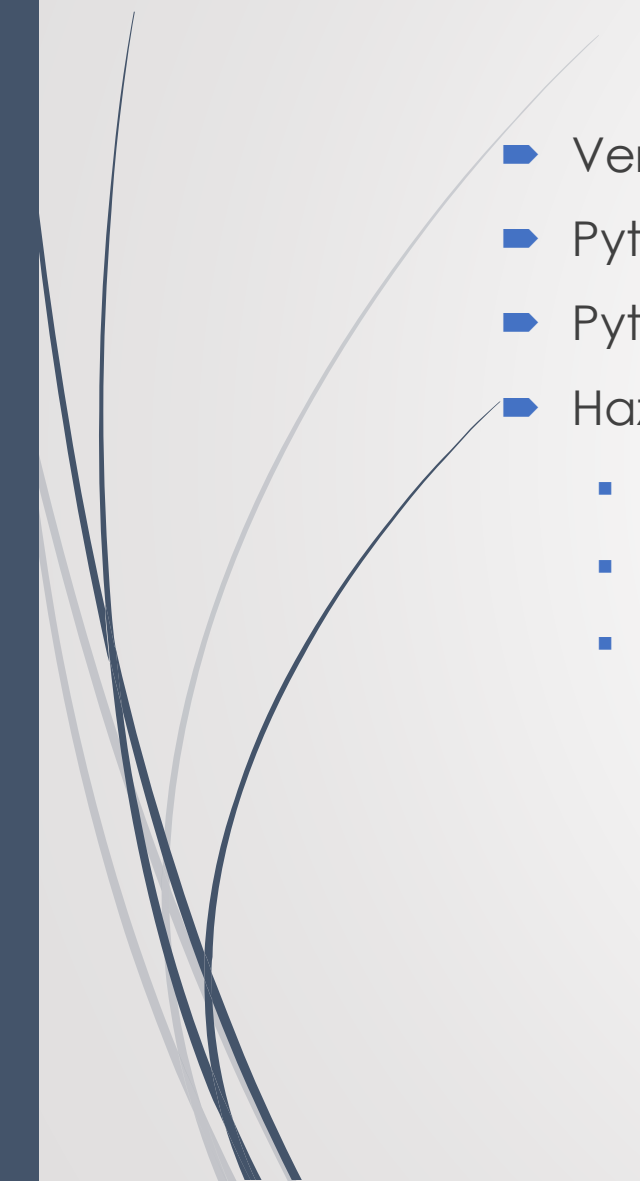


BSM422 Bilgisayarla Görme ve Görüntüleme Teknikleri

Dr. Öğr. Üyesi Mehmet Zahid YILDIRIM

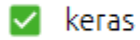


Bu ders;

- Veri Artırım Teknikleri
 - Python ile Veri Artırımı
 - Python ile Model Oluşturma
 - Hazır modeller (Pre-Trained)
 - ResNet50
 - VGG16
 - VGG19
- 

Gerekli Kurulumlar

- **TensorFlow**, Google tarafından geliştirilen açık kaynaklı bir makine öğrenmesi ve derin öğrenme kütüphanesidir. Büyük ölçekli sayısal hesaplamaları veri akış grafikleri üzerinden gerçekleştirir ve CPU, GPU hatta TPU gibi donanımlarda yüksek performansla çalışabilir.



keras



Deep learning for humans

- **Keras** ise derin öğrenme modellerini daha kolay ve hızlı geliştirmek için tasarlanmış yüksek seviyeli bir API'dir.



tensorflow



Tensorflow is an end-to-end open source platform for machine learning.

- Günümüzde Keras, TensorFlow ile entegre çalışarak (**tensorflow.keras**) onun hesaplama gücünü kullanır.

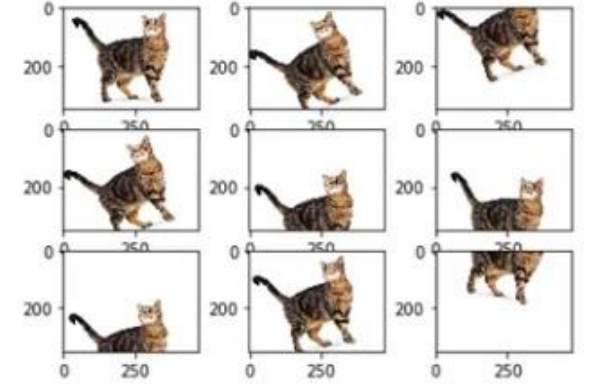
Veri Artırma (Data Augmentation)

- **Data Augmentation**, mevcut veri setini yapay olarak genişletmek için veriler üzerinde çeşitli dönüşümler uygulanmasıdır.

- Özellikle görüntü verilerinde;

- Rotation (Döndürme),
- Flip (Çevirme),
- Crop (Kırpma),
- Zoom (Yakınlaştırma)

gibi işlemlerle yeni örnekler üretilir.



- Amaç, modelin daha fazla çeşitlilik görmesini sağlayarak **overfitting**'i **azaltmak** ve genelleme performansını artırmaktır.

- ****Veri artırma → tüm veride mi.. / Belirli bir sınıf için mi.. ??****

Python ile Veri Artırma

Kütüphane

```
[1]: # Library  
  
from keras.preprocessing.image import load_img, img_to_array
```

Görüntüyü import etmek için..

Görüntü formatını Array olarak dönüştürmek için..

Python ile Veri Artırma

Verileri Hazırlama

```
[2]: # Veri Ön Hazırlık  
# PIL --> Array --> Reshaped Array
```

```
img = load_img("dataset/penguins.jpg")  
type(img) # Python Imaging Library
```

PIL.JpegImagePlugin.JpegImageFile

```
img_array = img_to_array(img)  
print(img_array.shape)
```

(192, 256, 3)

```
reshaped_img = img_array.reshape((1,) + img_array.shape)  
print(reshaped_img.shape) # görsel yığını oluşturma..
```

(1, 192, 256, 3)

Python ile Veri Artırma

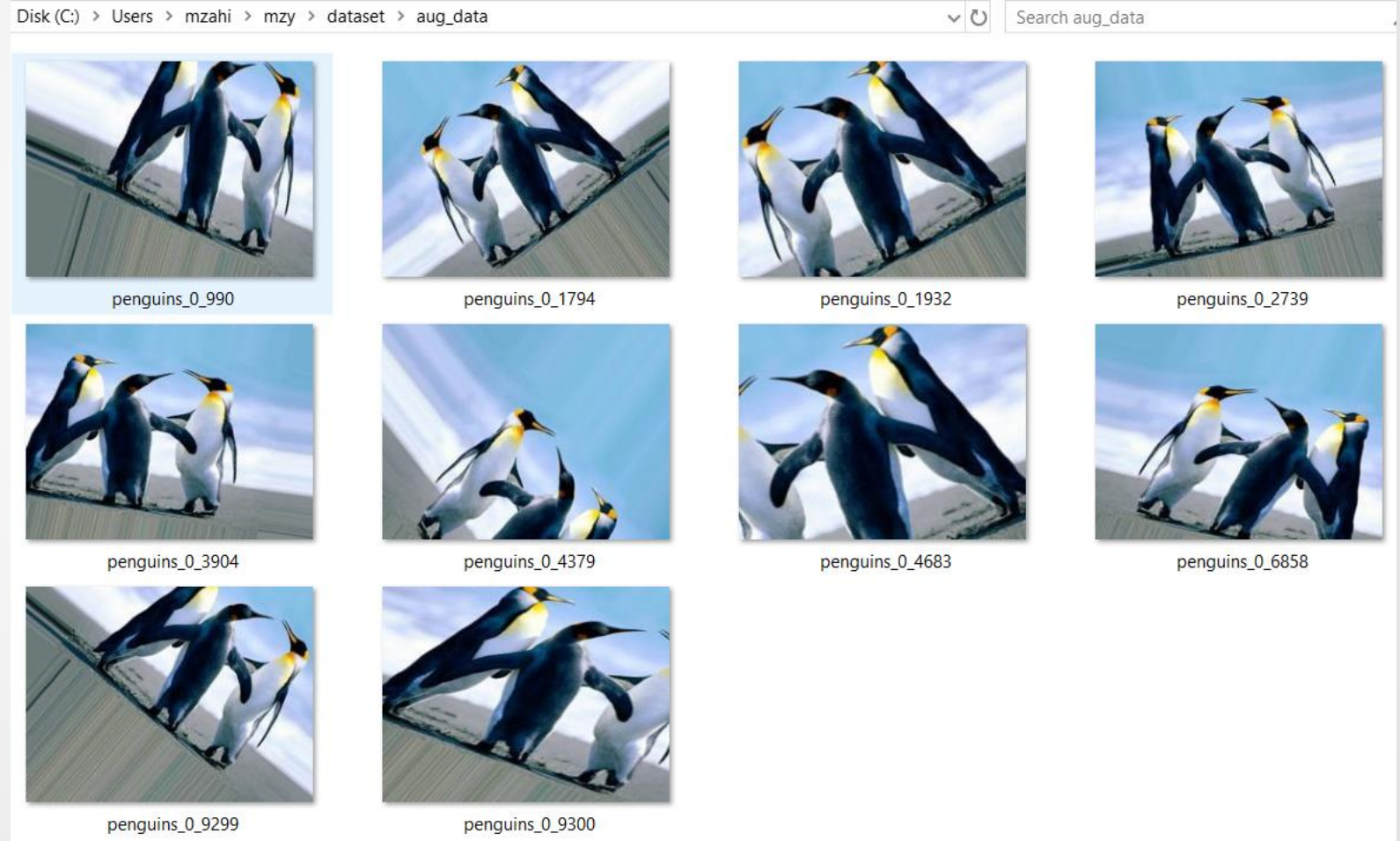
Görsel(ler) Üzerinden

```
[3]: # penguins.jpg görselinden "cnt" adedince artırım..  
# tek görsel üzerinden..  
from tensorflow.keras.preprocessing.image import ImageDataGenerator  
  
data_generator = ImageDataGenerator(rotation_range=45,  
                                   width_shift_range=0.2,  
                                   height_shift_range=0.2,  
                                   zoom_range=0.3,  
                                   horizontal_flip=True,  
                                   fill_mode="nearest")  
  
cnt = 0  
for i in data_generator.flow(reshaped_img,  
                             batch_size=1,  
                             save_to_dir="dataset/aug_data",  
                             save_prefix="penguins",  
                             save_format="jpg"):  
  
    cnt += 1  
    if cnt == 10:  
        break
```



Python ile Veri Artırma

Görsel(ler) Üzerinden



Python ile Veri Artırma

Klasörler (Sınıflar) Kullanılarak

```
[4]: # birden fazla görsel (sınıf) ile veri artırımı..

data_generator = ImageDataGenerator(rotation_range=45,
                                     width_shift_range=0.2,
                                     height_shift_range=0.2,
                                     zoom_range=0.3,
                                     horizontal_flip=True,
                                     fill_mode="nearest")

cnt = 0
for i in data_generator.flow_from_directory("dataset/aug_class",
                                           save_to_dir="dataset/aug_class",
                                           batch_size=1,
                                           save_format="jpg"):

    cnt += 1
    if cnt == 10:
        break
```

Found 2 images belonging to 2 classes.

al Disk (C:) > Users > mzahi > mzy > dataset > aug_class



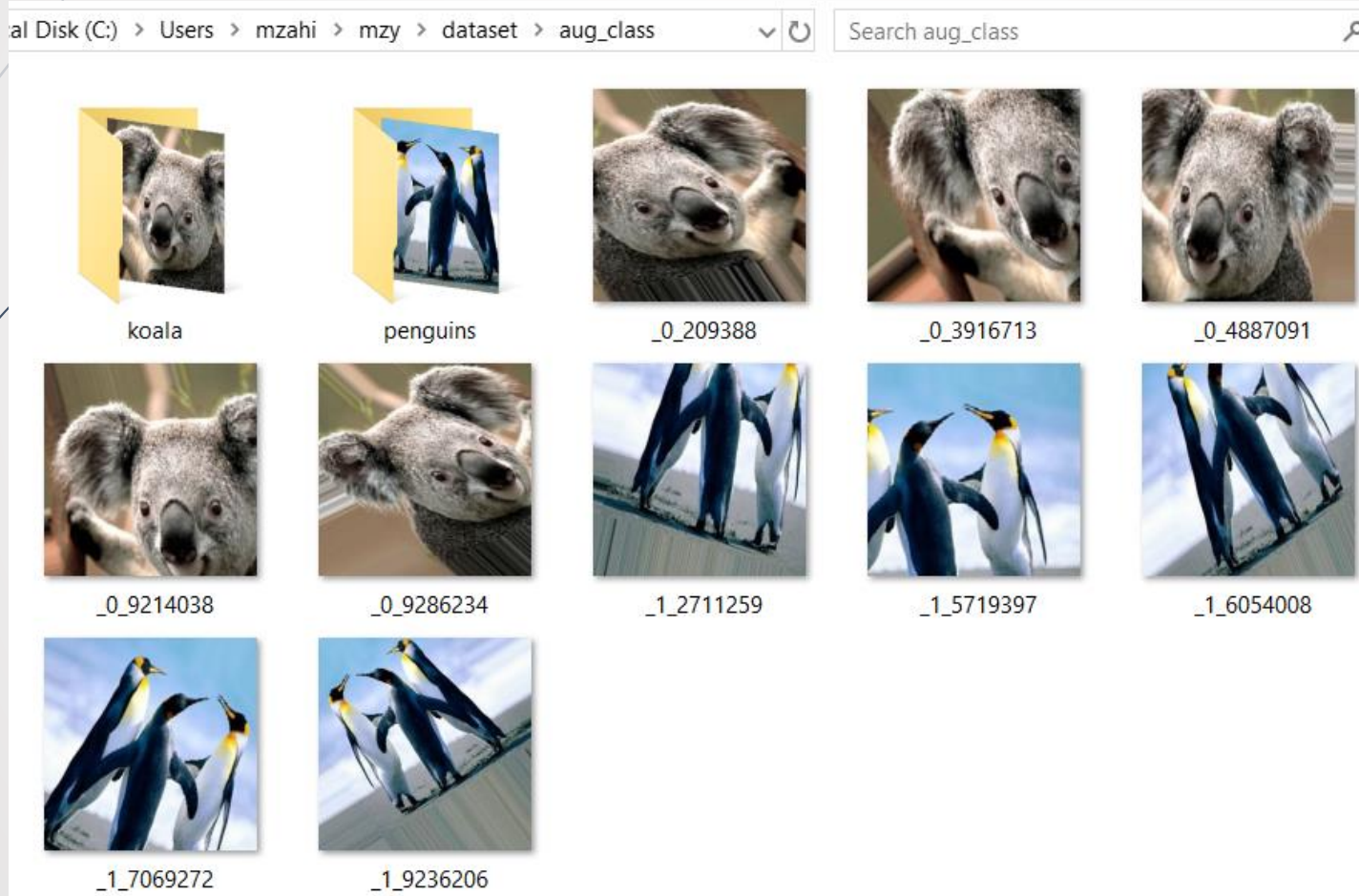
koala



penguins

Python ile Veri Artırma

Klasörler (Sınıflar) Kullanılarak



Python ile Görüntü Sınıflandırma

Proje Hazırlık (Keras)

Bir görüntü sınıflandırma projesinde;

1. Veri Seti

- Normalizasyon (/255)
- Augmentation (sadece Train'e – *imbalance* oluşmamalı)

2. Klasörlerin Hazırlanması (Aynı veriyi farklı klasörde kullanma!)

- Train
- Validation
- Test

3. Model Oluşturma

4. Eğitim

5. Test

Veriyi Böl

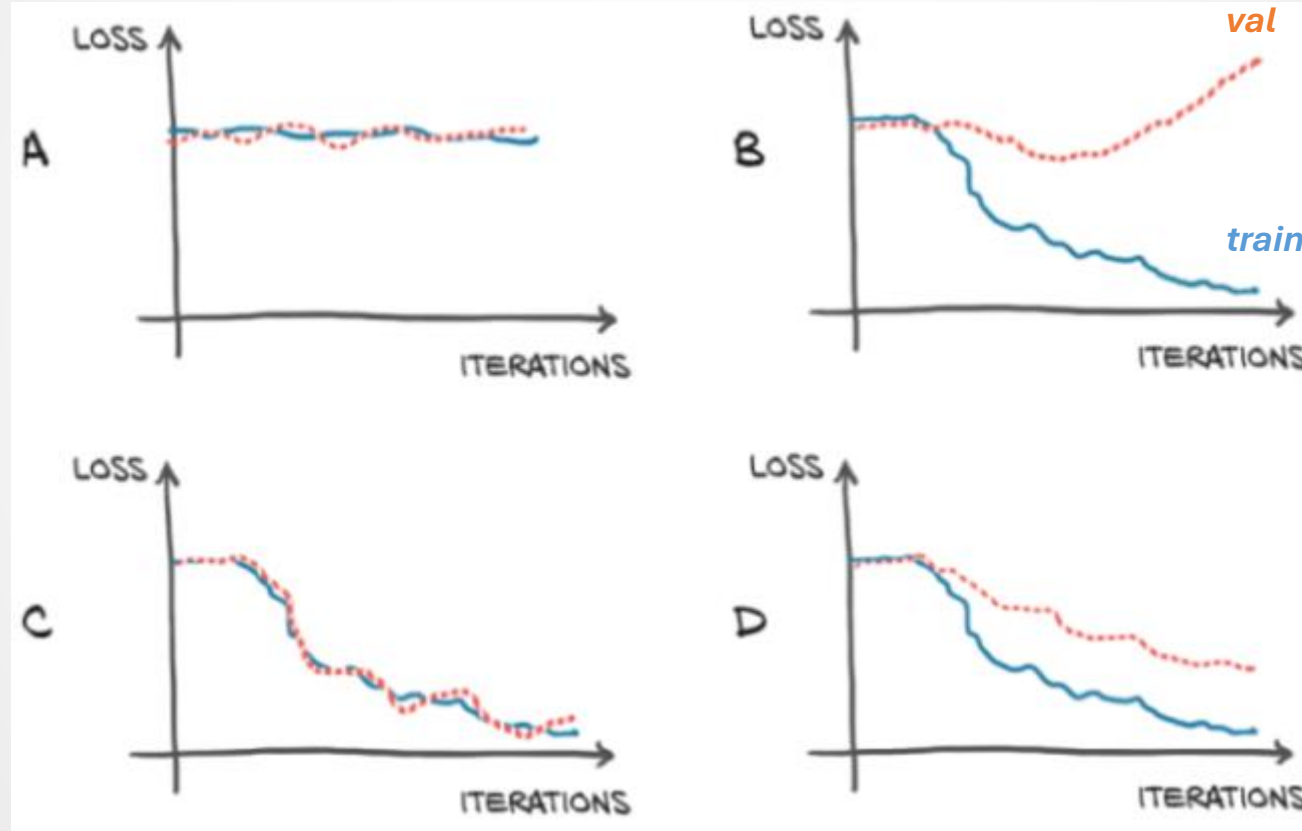
- %70 → train
- %15 → validation
- %15 → test

Örnek Klasör Yapısı

```
dataset/  
  train/  
    cat/  
      img1.jpg  
      img2.jpg  
    dog/  
      img3.jpg  
      img4.jpg  
  val/  
    cat/  
    dog/  
  test/  
    cat/  
    dog/
```

Python ile Görüntü Sınıflandırma

Model Sonuç Analizi



Python ile Hazır (Pre-Trained) Modeller

Resnet50

```
[1]: # Library
from tensorflow.keras.applications.resnet50 import ResNet50
from tensorflow.keras.applications.resnet50 import preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image
import numpy as np
```

- **ResNet50** → Önceden eğitilmiş CNN modeli (ImageNet üzerinde eğitilmiş)
- **preprocess_input** → Görüntüyü modele uygun formata çevirir
- **decode_predictions** → Model çıktısını insan tarafından anlaşılır etiketlere çevirir
- **image** → Görüntü yükleme ve işleme işlemleri
- **numpy** → Sayısal işlemler

```
[2]: model = ResNet50(weights = "imagenet")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50\_weights\_tf\_dim\_ordering\_tf\_kernels.h5
102967424/102967424 ————— 5s 0us/step
```

- Model, **ImageNet veri seti** üzerinde eğitilmiş ağırlıklarla yüklenir.
- **ImageNet** → 1000 farklı sınıf içerir (kedi, köpek, araba, kuş vb.)

Python ile Hazır (Pre-Trained) Modeller

Resnet50

```
[3]: test_path = "dataset/penguins.jpg"
img = image.load_img(test_path, target_size=(224,224))
img = image.img_to_array(img)
img = np.expand_dims(img, axis=0)
img = preprocess_input(img)

prediction = model.predict(img)
```

1/1 ————— 2s 2s/step

Görüntü → **NumPy array** formatına çevrilir.
(224, 224, 3)

Modele tek bir görüntü yerine batch (mini veri grubu) vermek gerekir. Bu yüzden boyut eklenir.
(1, 224, 224, 3)

Model, görüntüyü analiz eder.
Çıktı: 1000 sınıfa ait olasılık değerleri

```
[4]: prediction = decode_predictions(prediction, top=3)[0]
prediction
```

En yüksek olasılığa sahip **ilk 3** tahmin alınır.

```
[4]: [('n02056570', 'king_penguin', np.float32(0.9999902)),
      ('n02071294', 'killer_whale', np.float32(2.0849036e-06)),
      ('n02058221', 'albatross', np.float32(1.0218791e-06))]
```


Python ile Hazır (Pre-Trained) Modeller

VGG16

```
# Library
from tensorflow.keras.applications.vgg16 import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image
import numpy as np

# Load Weights
model = VGG16(weights = "imagenet")

# -----
test_path = "dataset/penguins.jpg"
img = image.load_img(test_path, target_size=(224,224))
img = image.img_to_array(img)
img = np.expand_dims(img, axis=0) # 0. indise boyut ekle
img = preprocess_input(img)

# Predict with Model
prediction = model.predict(img)
prediction = decode_predictions(prediction, top=3)[0]
prediction
```

```
[('n02056570', 'king_penguin', np.float32(1.0)),
 ('n01828970', 'bee_eater', np.float32(1.0854677e-10)),
 ('n01843383', 'toucan', np.float32(5.971239e-12))]
```

Python ile Hazır (Pre-Trained) Modeller

VGG19

```
# Library
from tensorflow.keras.applications.vgg19 import VGG19
from tensorflow.keras.applications.vgg19 import preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image
import numpy as np

# Load Weights
model = VGG19(weights = "imagenet")

# -----
test_path = "dataset/penguins.jpg"
img = image.load_img(test_path, target_size=(224,224))
img = image.img_to_array(img)
img = np.expand_dims(img, axis=0) # 0. indise boyut ekle
img = preprocess_input(img)

# Predict with Model
prediction = model.predict(img)
prediction = decode_predictions(prediction, top=3)[0]
prediction
```

```
[('n02056570', 'king_penguin', np.float32(1.0)),
 ('n01828970', 'bee_eater', np.float32(5.7412473e-12)),
 ('n01843383', 'toucan', np.float32(1.1198691e-13))]
```